Sparse and robust geometric twin support vector machine via asymmetric RoBoSS loss function

Kai Qi[a], Xinji Huang[b], Hongchun Wang[b,*]

[a]*National Center for Applied Mathematics in Chongqing, Chongqing Normal University, Chongqing, 401331, China.*
[b]*School of Mathematical Sciences, Chongqing Normal University, Chongqing, 401331, China.*

Abstract

In real-world scenarios, the training data usually contains redundant features, label noise and feature noise, which provide severe challenges for the efficiency of machine learning methods. Since standard support vector machine (SVM) adopts l_2-norm penalty and hinge loss function, it lacks the ability of selecting significant features and is sensitive to noise. To address these issues, this paper proposes a novel asymmetric, robust, bounded, sparse and smooth (aR) loss function for l_1-norm penalized geometric twin SVM (aRSGTSVM) to handle classification and regression tasks. The l_1-norm penalty can achieve the feature selection. The proposed aR loss function can not only effectively mitigate the impact of label noise, but also significantly enhance the stability to resampling noise, i.e., the zero-mean feature noise around the boundary hyperplanes. Furthermore, a statistical analysis of the robustness of aRSGTSVM was also conducted using the influence function. Since aRSGTSVM involves nonconvex and nonsmooth optimization, we develop a fast and stable proximal gradient descent based solving algorithm. Compared with related state-of-the-art methods, experimental results demonstrate the superiority of the proposed aRSGTSVM on both synthetic and UCI datasets. Furthermore, we apply aRSGTSVM to index tracking tasks, where results for tracking the different indices in the China stock market show that it can achieve satisfactory performance.

*Corresponding author. *Email: wanghc@cqnu.edu.cn*
Email addresses: qikai@cqnu.edu.cn (Kai Qi), 2024110510040@stu.cqnu.edu.cn (Xinji Huang)

Keywords: Feature selection, robustness, geometric twin classification and regression, asymmetric RoBoSS loss function, nonconvex and nonsmooth optimization

1. Introduction

Twin support vector machine (TSVM) [1] is one of the well-known variants of the support vector machine, which has experienced prosperous developments in the past decade and has been widely applied into different fields, such as digit recognition [2], medical diagnosis [3, 4] and bioanalysis [5]. From a geometric perspective, TSVM aims to find a pair of nonparallel separating hyperplanes, where each hyperplane is required to be close to the samples of one class while being far from those of the other class. This makes TSVM more flexible and efficient than standard SVM. Moreover, similar to SVM, the standard TSVM can also be explained from a statistical point of view, i.e., it fits the "loss + penalty" regularization framework [3, 6]. Specifically, most of the existing TSVM adopt hinge loss and l_2-norm penalty. However, hinge loss is proved to be sensitive to label noise [7, 8] and resampling noise (zero-mean feature noise around the boundary hyperplanes) [9]. The l_2-norm penalty lacks the ability to perform feature selection [10, 11]. Therefore, there remains significant room for improving the efficiency of TSVM in handling complex real-world problems.

The label noise sensitivity of SVMs and TSVMs stems mainly from the upper-unbounded nature of the hinge loss. Specifically, label noise usually lies near or even on the wrong side of the separating hyperplane. For such samples, the losses can be quite large due to the unboundedness of hinge loss function. Consequently, the final obtained separating hyperplane tends to be deviated by label noise by the minimization of the optimization objective. To address this issue, researchers have turned to nonconvex loss functions to enhance robustness against label noise. By considering the difference between the logistic loss and its shifted version, Krause and Singer [12] proposed logistic difference (LD) loss for SVM to reduce the influence of label noise. Wu and Liu [7] integrated a truncated hinge loss, so-called ramp loss, with SVM to enhance the robustness. Later, Liu et al. [13] combined the ramp loss with TSVM to propose robust nonparallel SVM (RNPSVM). Wang et al. [14] constructed a capped

l_1-norm loss for robust TSVM. Wang et al. [8] designed a new truncated squared loss to obtain a robust SVM (L_{tsl}-SVM). In addition to truncation, correntropy is also an effective approach of designing robust losses [15, 16]. Xu et al. [17] developed the idea of correntropy to SVM and proposed rescaled hinge loss SVM, which can alleviate the disturbance of outliers. Xu et al. [18] studied the correntropy based loss (C-loss) for least squares SVM. Ma et al. [19] constructed a novel adaptive capped loss derived from correntropy to enhance TSVM's resistance to label noise. Recently, Akhtar et al. [20] introduced a novel robust, bounded, sparse and smooth (RoBoSS) loss function for SVM, to mitigate the impact of label noise. However, RoBoSS loss ignores the perturbation of resampling noise and the performance needs further improving.

Regarding resampling noise, Huang et al. [9] first demonstrated that hinge loss-based separating hyperplanes are sensitive to zero-mean feature noise near the boundary. Consequently, solutions derived from resampling procedures, such as K-fold cross-validation lack stability. Inspired by statistical quantiles, they introduced the pinball loss to SVM (PinSVM) to enhance its resampling stability. The authors [21] further extended this idea to squared loss, developing an asymmetric least squares SVM with a smooth, easily optimized objective function. A key limitation of PinSVM is the loss of sample sparsity. Specifically, all training samples become support vectors. This drawback can increase the training burden [9]. To encourage the sparsity, Shen et al. [22] proposed a truncated pinball loss, which can provide a flexible trade-off between sparsity and feature noise insensitivity. Motivated by quantiles and correntropy, Yang and Dong [23] proposed a generalized quantile loss. Based on the least squares SVM, He et al. [24] established an asymmetric kernel-based learning classifier.

For redundant features, they often provide irrelevant or even misleading information for the modelling process. As a result, the predicting performance may degrade. Zhu et al. [10] replaced l_2-norm penalty by l_1-norm penalty to propose 1-norm SVM, which can select significant features and remove redundant features, simultaneously. Ikeda and Murata [25] investigated the geometrical properties of l_p-norm penalized ν-SVM, where $1 \leq p \leq \infty$. Zhu et al. [11] integrated elastic net with SVM to construct doubly regularized SVM (DrSVM). In many real-world scenarios, input features exhibit a grouped structure. Consequently, there is greater interest in performing feature

selection or elimination at the group level rather than on an individual basis. To achieve group selection, Zou and Yuan [26] applied the infinity norm for each group of features to propose F_∞-nrom SVM. Gao et al. [27] employed l_1-norm penalty for least squares TSVM to automatically select input features. Moosaei and Hladík [28] proposed l_p-norm ($0 < p < 1$) least squares twin multi-class SVM. With $p \geq 1$, Xie et al. [29] designed Laplacian l_p-norm least squares TSVM.

Despite advances in robust SVM methods, existing approaches suffer critical limitations that hinder practical use. Label-noise-robust losses such as ramp loss, capped loss, and RoBoSS typically ignore resampling noise, while resampling-stable variants like pinball loss remain sensitive to label noise. Most robust losses either sacrifice sparsity or introduce nonconvexity with hyperparameters lacking theoretical tuning guidance. Moreover, few prior methods address label noise, resampling instability, and redundant feature interference simultaneously within a unified framework. Notably, RoBoSS lacks resampling-noise resistance and its feature-selection performance remains unexplored. In this paper, to address the aforementioned issues, we propose a novel asymmetric RoBoSS loss-based sparse geometric TSVM (aRSGTSVM). The proposed asymmetric RoBoSS loss is smooth and bounded, which can mitigate the impact of label noise and resampling noise, simultaneously. Moreover, aRSGTSVM incorporates an l_1-norm penalty to enable feature selection.

In short, the main contributions are summarized as follows:

1) We propose a new asymmetric and robust (aR) loss function. Especially, compared with RoBoSS loss function, the proposed aR loss function can not only effectively mitigate the impact of label noise, but also enhance the stability to the zero-mean noise near the boundary hyperplanes (so-called resampling noise).

2) We employ influence function to demonstrate the robustness of aR loss function. Although there is extensive literature on enhancing SVM robustness by constructing non-convex functions, few studies theoretically elucidate the robustness of the constructed functions. From a statistical perspective, we further proved that the influence function of aR loss is bounded, thereby providing a theoretical guarantee of its robustness.

3) Based on the aR loss and l_1-norm penalty, the aRSGTSVM models are proposed

for robust classification and regression learning problems (In the following content, we refer to the regression model as aRSGTSVR to avoid confusion). In addition to robustness, the proposed aRSGTSVM and aRSGTSVR can also reduce the influence of noise features.

4) To optimize the nonconvex and nonsmooth aRSGTSVM and aRSGTSVR optimization problems, we design an efficient algorithm based on the proximal gradient descent algorithm, termed as iPiano algorithm. The implemented algorithm is demonstrated to be fast and stable, and is applicable to high-dimensional learning problems.

5) A lot of numerical studies on synthetic and benchmark datasets demonstrate that the proposed aRSGTSVM and aRSGTSVR are robust against label noise and resampling noise, while maintaining excellent feature selection capability in high-dimensional settings.

6) To further validate the generalization capability, we apply the model into index tracking task. The results show that aRSGTSVR delivers consistently superior performance across different underlying indices.

The rest of paper is organized as follows: Section 2 introduces the related work of this paper. Section 3 proposes the aR loss function and applies it to both classification and regression problems. Section 4 presents a lot of experimental results of the proposed models on artificial datasets, UCI datasets and real-world stock datasets. Finally, in Section 5, we revisit the main contributions of this paper and provide a concluding remark.

2. Preliminaries and related work

First, we define some necessary notation. Consider a supervised learning data $\{(x_i^T, y_i)^T\}_{i=1}^n$ with n samples and p features, where $x_i = (x_{i1}, x_{i2}, \cdots, x_{ip})^T \in \mathbb{R}^p$ is the i-th sample. Note that $y_i \in \{\pm 1\}$ means a binary classification problem, while $y_i \in \mathbb{R}$ means a regression problem. Furthermore, let $Y = (y_1, y_2, \cdots y_n)^T \in \mathbb{R}^n$ and $X = (x_1, x_2, \cdots x_n)^T \in \mathbb{R}^{n\times p}$. The positive and the negative samples are reorganized as X_+ and X_-, respectively. Unless otherwise specified, all the vectors mentioned below are in column-form and the norm defaults to the l_2-norm.

2.1. ENNHSVM

For the standard TSVM, its training process does not fully consider the core requirement of comparing distances to each separating hyperplane in the prediction stage, leading to an inconsistency between the training and prediction processes. As [30] pointed out, this inconsistency can possibly reduce the prediction performance, especially in the datasets with heteroscedastic noise.

Recently, Qi and Yang [3] proposed a novel consistent ENNHSVM, which performs better than the standard TSVM. Specifically, ENNHSVM pursues two nonparallel hyperplanes by optimizing the following problems:

$$\begin{aligned}
\min & \frac{c_1}{2}\left(\|X_+w_+ + e_+b_+\|^2 + \|X_-w_- + e_-b_-\|^2\right) \\
& + \frac{1}{2}\left(\|w_+\|^2 + b_+^2 + \|w_-\|^2 + b_-^2\right) \\
& + \frac{c_2}{2}\left(\|\xi_+\|^2 + \|\xi_-\|^2\right) + c_3\left(e_+^T\xi_+ + e_-^T\xi_-\right) \\
\text{s.t.} & \begin{cases} X_+w_+ + e_+b_+ + X_+w_- + e_+b_- \geq e_+ - \xi_+, \xi_+ \geq 0, \\ X_-w_- + e_-b_- + X_-w_+ + e_-b_+ \leq \xi_- - e_-, \xi_- \geq 0, \end{cases}
\end{aligned} \tag{1}$$

where $w_\pm$ and $b_\pm$ are the normal vectors and intercepts of the positive and negative hyperplanes. c_i $(i = 1, 2, 3)$ are positive tuning parameters, $\xi_\pm$ are slack variables and $e_\pm$ are two vectors with all elements equaling to one.

After obtaining $w_\pm$ and $b_\pm$, for a given new sample x_{new}, its label can be classified via the following decision function:

$$f(x_{\text{new}}) = \text{sign}\Big(x_{\text{new}}^T(w_+ + w_-) + (b_+ + b_-)\Big), \tag{2}$$

where sign$(\cdot)$ denotes the sign function.

Although ENNHSVM is consistent and demonstrated to be efficient, it cannot perform feature selection and exhibits poor performance in the presence of redundant variables. Moreover, its application in regression has not yet been explored in depth.

2.2. RoBoSS loss for SVM

To reduce the impact of label noise, recently, Akhtar et al. [20] proposed a robust, bounded, sparse and smooth loss (RoBoSS). They incorporated the RoBoSS loss into the SVM framework, resulting in a novel robust RoBoSS-SVM.

In detail, RoBoSS loss is defined as

$$L_{RoBoSS}(u) = \begin{cases} \lambda(1 - (au + 1)\exp(-au)), & u > 0, \\ 0, & u \leq 0, \end{cases} \tag{3}$$

where a is the shape parameter and λ is the boundary parameter. The primal problem of RoBoSS-SVM can be formulated as

$$\min_{w,b} \frac{1}{2}\|w\|^2 + \frac{C}{n}\sum_{i=1}^{n} L_{\text{RoBoSS}}\left(1 - y_i\left(w^T x_i + b\right)\right). \tag{4}$$

where w and b are the normal vector and intercept of the separating hyperplane. $C > 0$ is the tuning parameter.

3. The proposed aRSGTSVM(R)

As discussed earlier, although the RoBoSS loss is robust to noise, it exhibits instability to resampling. In this chapter, an asymmetric version of the RoBoSS loss, termed aR loss, is proposed to address more complex noise in high-dimensional learning. Specifically, Section 3.1 introduces the aR loss and investigates its theoretical properties. Section 3.2 presents the aRSGTSVM method for classification problems, while Section 3.3 proposes the aRSGTSVR model for regression scenarios. Finally, Section 3.4 provides the corresponding optimization algorithm.

3.1. asymmetric RoBoSS loss

Although the RoBoSS loss demonstrates considerable robustness to label noise, Huang et al. [31] point out that in many practical problems, beyond outliers, high-dimensional complex data may also be affected by other types of noise, such as resampling noise or zero-mean noise near the bounding hyperplane. However, one-sided

loss functions like the hinge loss and RoBoSS loss struggle to effectively mitigate the impact of such noise, leading to unstable model performance and leaving room for improvement. Therefore, to further enhance the capability of the RoBoSS loss in handling complex data, we propose a new loss function named aR, which is defined as follows:

$$L_{aR}(u) = \begin{cases} \lambda(1 - (au + 1)\exp(-au)), & u > 0, \\ \tau\lambda(1 - (au^2 + 1)\exp(-au^2)), & u \leq 0, \end{cases} \tag{5}$$

where the shape parameter $a > 0$, the margin parameter $\lambda > 0$, and the parameter $\tau \in [0, 1]$ control the asymmetry of the loss function, thereby enhancing the model's robustness to resampling noise.

Next, we elaborate on the properties of the proposed aR loss function.

Property 1. The aR loss function is C^1-smooth.

Proof. First, by (5), it follows that

$$\nabla L_{aR}(u) = \begin{cases} \lambda a^2 u \exp(-au), & u > 0, \\ 2\tau\lambda a^2 u^3 \exp(-au^2), & u \leq 0, \end{cases} \tag{6}$$

Thus, we can deduce that $\nabla L_{aR}(0^+) = 0$ and $\nabla L_{aR}(0^-) = 0$. According to (5), we have $\lim_{u\to 0^+} L_{aR}(u) = \lim_{u\to 0^-} L_{aR}(u) = L_{aR}(0) = 0$, which implies that $L_{aR}(u)$ is continuous at $u = 0$. Consequently, the aR loss is a C^1-smooth function. □

Property 2. The aR loss function is bounded.

Proof. For $u > 0$, we have $\nabla L_{aR}(u) = \lambda a^2 u \exp(-au) > 0$, which implies that $L_{aR}(u)$ is strictly monotonically increasing on $(0, +\infty)$. For $u < 0$, we have $\nabla L_{aR}(u) = 2\tau\lambda a^2 u^3 \exp(-au^2) < 0$, which implies that $L_{aR}(u)$ is strictly monotonically decreasing on $(-\infty, 0)$. According to the monotonicity above, the function reaches its global minimum at $u = 0$, and $L_{aR}(0) = 0$, which means that the loss function is lower bounded by 0.

Next, we analyze the upper bound of the function by calculating the limits at infinity:

$$\lim_{u\to+\infty} L_{aR}(u) = \lambda, \quad \lim_{u\to-\infty} L_{aR}(u) = \tau\lambda. \tag{7}$$

Since $0 < \tau \leq 1$, we have $\tau\lambda \leq \lambda$. Combined with monotonicity on both intervals, all function values satisfy

$$0 \leq L_{aR}(u) \leq \lambda, \quad \forall u \in \mathbb{R}. \tag{8}$$

Therefore, the aR loss function is both lower bounded and upper bounded, namely, it is bounded on $\mathbb{R}$. □

Property 3. The aR loss is robust to label noise (outliers), which can be theoretically guaranteed by influence function.

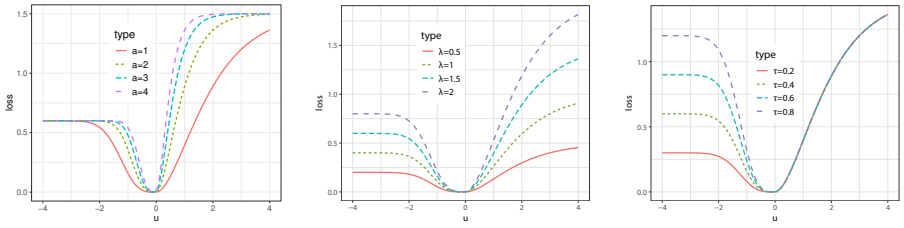
Proof. Hampel introduced the influence function [32], which is mainly used to measure the stability of an estimator when subjected to infinitesimal contamination. For an ideal roust loss function, the influence function of the induced estimator is bounded.

Following [33] and by (6), we have that ∇l_{aR} achieves its maximum at $u = \frac{1}{a}$ for $u > 0$, while it achieves its maximum at $u = -\sqrt{\frac{3}{2a}}$ for $u \leq 0$. Therefore, it follows that

$$|\mathrm{IF}(u)| \leq \max\left\{\frac{\lambda a}{\mathrm{e}}, \frac{3\tau\lambda\sqrt{6a}}{2\mathrm{e}^{3/2}}\right\} < \infty, \quad \forall u \in \mathbb{R}. \tag{9}$$

Therefore, according to the influence function theory, for the SVM model designed based on the aR loss, even if the loss u caused by label noise is extremely large, their impact on the model is still limited. This theoretically guarantees the robustness of the aR loss.

Figure 1 illustrates the various forms of the aR loss function under different parameters. Combined with the preceding propositions, the asymmetric RoBoSS (aR) loss function is a bounded, asymmetric, smooth, and non-convex function. Consequently, compared to the original RoBoSS loss, our proposed loss function is not only robust to outliers but also resilient to zero-mean feature noise, enabling it to handle more complex problems.



(a) Different a values (b) Different λ values (c) Different τ values

Figure 1: Illustrations of different parameter values of the proposed aR loss function

3.2. aR loss-based sparse geometric TSVM

In this section, we mainly study the classification problems. Since the proposed aR loss can effectively mitigate the impact of outliers and strengthen the stability to resampling noise, simultaneously. We propose a novel sparse and robust geometric twin support vector machine(aRSGTSVM), which integrates the aR loss function with ENNHSVM, thereby inheriting the consistency properties of ENNHSVM. The resulting expression is as follows:

$$\begin{aligned}
\min \quad & \frac{c_1}{2}\left(\|X_+w_+ + e_+b_+\|^2 + \|X_-w_- + e_-b_-\|^2\right) \\
& + \lambda\left(\|w_+\|_1 + |b_+| + \|w_-\|_1 + |b_-|\right) \\
& + c_2\left(\sum_{i\in N_+} L_{aR}\left((\xi_+)_i\right) + \sum_{i\in N_-} L_{aR}\left((\xi_-)_i\right)\right) \\
\text{s.t.} \quad & \begin{cases} X_+w_+ + e_+b_+ + X_+w_- + e_+b_- \geq e_+ - \xi_+, \\ X_-w_- + e_-b_- + X_-w_+ + e_-b_+ \leq \xi_- - e_-, \\ \xi_+ \geq 0, \xi_- \geq 0, \end{cases}
\end{aligned} \tag{10}$$

where $N_\pm$ denotes by the index sets of positive and negative samples. $(\xi_\pm)_i$ means the i-th element of the slack variables. Other notations are similar to ENNHSVM.

In contrast to ENNHSVM, our proposed classifier replaces the loss function in the original model with the aR loss and incorporates an l_1-norm penalty to equip the model with feature selection capability. The core idea of the proposed aRSGTSVM is outlined as follows:

1) Minimizing the first term of the objective of (10) forces each hyperplane to be closer to its corresponding class samples and farther from the other class, enhancing inter-class discrimination.

2) The second term $||cdot||_1$ induces sparsity in the model solution, enabling effective variable selection and handling of high-dimensional complex problems.

3) The third term employs the aR loss function, whose inherent properties provide robustness against label noise and resampling noise, thereby improving the model's generalization performance.

To simplify subsequent derivations, define the following notations: $\tilde{X}_+ = (X_+, e_+)$, $\tilde{X}_- = (X_-, e_-)$, $\tilde{w}_\pm = (w_\pm^T, b_\pm)^T$ and $\tilde{w} = (\tilde{w}_+, \tilde{w}_-)$. Furthermore, let

$$A = \begin{pmatrix} \tilde{X}_+^T \tilde{X}_+ & 0 \\ 0 & \tilde{X}_-^T \tilde{X}_- \end{pmatrix}, \quad B = \begin{pmatrix} \tilde{X}_+ & \tilde{X}_+ \\ -\tilde{X}_- & -\tilde{X}_- \end{pmatrix}, \quad e = \begin{pmatrix} e_+ \\ e_- \end{pmatrix}, \quad \xi = \begin{pmatrix} \xi_+ \\ \xi_- \end{pmatrix}. \tag{11}$$

Then, the aRSGTSVM problem (10) can be rewritten as:

$$\begin{aligned} &\min \frac{c_1}{2} \tilde{w}^T A \tilde{w} + \lambda ||\tilde{w}||_1 + \frac{c_2}{2} ||\xi||^2 \\ &\text{s.t.} \begin{cases} B\tilde{w} \geq e - \xi, \\ \xi \geq 0. \end{cases} \end{aligned} \tag{12}$$

which is equivalent to the following "loss + penalty" form, i.e.,

$$\min_{\tilde{w}} \frac{c_1}{2} \tilde{w}^T A \tilde{w} + c_2 \sum_{i=1}^{n} L_{aR}(1 - b_i^T \tilde{w}) + \lambda ||\tilde{w}||_1. \tag{13}$$

where b_i represents the i-th row vector of matrix B.

3.3. aR loss-based sparse geometric TSVR

To extend the application scope from discrete classification to continuous prediction, we apply aRSGTSVM to regression problems and propose aR loss-based sparse geometric TSVR (aRSGTSVR).

Using the idea of TSVR [34], let $X_+ = (X_+, Y + \varepsilon_1 e_+)$, $X_- = (X_-, Y - \varepsilon_2 e_-)$, $w_+ = \left(w_+^T, \eta_+\right)^T$, $w_- = \left(w_-^T, \eta_-\right)^T$, where $\varepsilon_1, \varepsilon_2 > 0$ are tuning parameters. We first transform (10) into the following regression problem:

$$
\begin{aligned}
\min\ & \frac{c_1}{2}\left(\|X_+ w_+ + \eta_+(Y + \varepsilon_1 e_+) + e_+ b_+\|^2 + \|X_- w_- + \eta_-(Y - \varepsilon_2 e_-) + e_- b_-\|^2\right) \\
& + \lambda\left(\|w_+\|_1 + |\eta_+| + |b_+| + \|w_-\|_1 + |\eta_-| + |b_-|\right) \\
& + c_2\left(\sum_{i\in N_+} L_{aR}\left((\xi_+)_i\right) + \sum_{i\in N_-} L_{aR}\left((\xi_-)_i\right)\right) \\
\text{s.t.}\ & \begin{cases} X_+ w_+ + \eta_+(Y + \varepsilon_1 e_+) + e_+ b_+ + X_+ w_- + \eta_-(Y + \varepsilon_1 e_+) + e_+ b_- \geq e_+ - \xi_+, \\ X_- w_- + \eta_-(Y - \varepsilon_2 e_-) + e_- b_- + X_- w_+ + \eta_+(Y - \varepsilon_2 e_-) + e_- b_+ \leq \xi_- - e_- < 0, \\ \xi_+ \geq 0, \quad \xi_- \geq 0. \end{cases}
\end{aligned}
\tag{14}
$$

Analogously to the classification problem, we further define the following variables: $\tilde{X}_+ = (X_+, Y + \varepsilon_1 e_+, 1)$, $\tilde{X}_- = (X_-, Y - \varepsilon_2 e_-, 1)$, $\tilde{w}_+ = \left(w_+^T, \eta_+, b_+\right)^T$, $\tilde{w}_- = \left(w_-^T, \eta_-, b_-\right)^T$, $\tilde{w} = \left(\tilde{w}_+^T, \tilde{w}_-^T\right)^T$, as well as the corresponding matrices and vectors:

$$
A = \begin{pmatrix} \tilde{X}_+^T \tilde{X}_+ & O \\ O & \tilde{X}_-^T \tilde{X}_- \end{pmatrix},\ B = \begin{pmatrix} \tilde{X}_+ & \tilde{X}_+ \\ -\tilde{X}_- & -\tilde{X}_- \end{pmatrix},\ e = \begin{pmatrix} e_+ \\ e_- \end{pmatrix},\ \xi = \begin{pmatrix} \xi_+ \\ \xi_- \end{pmatrix}.
$$

We rewrite (14) as:

$$
\begin{aligned}
& \min \frac{c_1}{2}\tilde{w}^T A \tilde{w} + \lambda\|\tilde{w}\|_1 + \frac{c_2}{2}\|\xi\|^2 \\
& \text{s.t.} \begin{cases} B\tilde{w} \geq e - \xi, \\ \xi \geq 0. \end{cases}
\end{aligned}
\tag{15}
$$

Similar to the previous steps, we obtain the linear case of the aRSGTSVR model:

$$
\min \frac{c_1}{2}\tilde{w}^T A \tilde{w} + c_2 \sum_{i=1}^{n} L_{aR}\left(1 - b_i^T \tilde{w}\right) + \lambda\|\tilde{w}\|_1. \tag{16}
$$

By utilizing the feature mapping $\mathbf{\Phi}(\cdot)$ that maps raw input space into a high dimensional Reproducing Kernel Hilbert Space, the proposed aRSGTSVM(R) model can be extended to the sparse kernel case. In fact, by only replacing X with $\mathbf{\Phi}(X)$, we derive

the aRSGTSVM(R) model for the sparse kernel scenario. Note that feature selection is one of the key focuses of this study, so we primarily consider the linear case

3.4. iPiano algorithm

Due to the non-convex aR loss function and non-smooth penalty term, Problem (10) constitutes a non-convex optimization problem. Conventional solution approaches typically employ the difference of convex functions (DC) algorithm or the half-quadratic (HQ) algorithm, which require iteratively solving a series of subproblems and incur substantial computational costs. Therefore, we opt to utilize the iPiano (Inertial Proximal Algorithm for Nonconvex Optimization) algorithm [35] for optimization, which offers advantages of rapid convergence and numerical stability, particularly suited for high-dimensional complex problems. The following derivation will be presented using the classification problem as an illustrative example.

3.4.1. iPiano for aRSGTSVM

The iPiano algorithm can solve optimization problems of the form:

$$\min_{x\in\mathbb{R}^n} h(x) = f(x) + g(x), \tag{17}$$

where g is convex (possibly nonsmooth), f is C^1-smooth (possibly nonconvex).

Thus, (10) can also be interpreted as having the form "$f + g$" :

$$\min_{\tilde{w}} \underbrace{\frac{c_1}{2}\tilde{w}^T A\tilde{w} + c_2 \sum_{i=1}^{n} L_{aR}\left(1 - b_i^T\tilde{w}\right)}_{f} + \underbrace{\lambda\|\tilde{w}\|_1}_{g}. \tag{18}$$

Let $u_i = 1 - b_i^T\tilde{w}$, then the objective function f is given by:

$$f(\tilde{w}) = \begin{cases} \frac{c_1}{2}\tilde{w}^T A\tilde{w} + c_2 \sum\limits_{i=1}^{n} \lambda\left(1 - (au_i + 1)\exp\left(-au_i\right)\right), & u_i > 0, \\ \frac{c_1}{2}\tilde{w}^T A\tilde{w} + c_2 \sum\limits_{i=1}^{n} \tau\lambda\left(1 - \left(au_i^2 + 1\right)\exp\left(-au_i^2\right)\right), & u_i \le 0. \end{cases} \tag{19}$$

Then ∇f is obtained:

$$\nabla f(\tilde{w}) = \begin{cases} c_1 A\tilde{w} - c_2 \sum_{i=1}^{n} a^2 \lambda b_i u_i \exp(-a u_i), & u_i > 0, \\ c_1 A\tilde{w} - c_2 \sum_{i=1}^{n} 2a^2 \tau \lambda b_i u_i^3 \exp(-a u_i^2), & u_i \leq 0. \end{cases} \tag{20}$$

Here, $L > 0$ denotes an upper bound constant for the Lipschitz continuity of ∇f, which satisfies the inequality

$$|\nabla f(\tilde{w}_1) - \nabla f(\tilde{w}_2)| \leq L\|\tilde{w}_1 - \tilde{w}_2\|, \quad \forall \tilde{w}_1, \tilde{w}_2. \tag{21}$$

Because

$$\nabla^2 f(\tilde{w}) = \begin{cases} c_1 A - c_2 \sum_{i=1}^{n} a^2 \lambda b_i b_i^T (-1 + a u_i) \exp(-a u_i), & u_i > 0, \\ c_1 A - c_2 \sum_{i=1}^{n} 2a^2 \tau \lambda b_i b_i^T \left(-3u_i^2 + 2a u_i^4\right) \exp(-a u_i^2), & u_i \leq 0, \end{cases} \tag{22}$$

we can get

$$L = c_1\|A\| + c_2 \lambda a^2 \max\left(1, \frac{2|\tau|}{a\sqrt{e}}\right) \sum_{i=1}^{n} \|b_i\|^2. \tag{23}$$

The general framework of the iPiano algorithm for the problem (17) is presented in the algorithm 1. In this work, we empirically set the maximum iteration number n_{iter} to 500 and the error threshold ε to 10^{-6}.

The proximal map is defined by

$$(I + \alpha \partial g)^{-1}(\hat{x}) := \arg\min_{x \in \mathbb{R}^n} \left\{ \alpha g(x) + \frac{1}{2}\|x - \hat{x}\|_2^2 \right\}, \tag{24}$$

where I is the identity map, $\alpha > 0$ is a given step size parameter, and g is a proper lower semicontinuous convex function whose specific definition is given in (18).

3.4.2. iPiano for aRSGTSVR

According to (16) and (13), the aRSGTSVR model can be reformulated into a form consistent with that of the aRSGTSVM. Thus, similarly to algorithm 1, the overall procedure for solving the aRSGTSVR algorithm is shown in algorithm 2.

Algorithm 1 iPiano for aRSGTSVM

Input: Set step size parameters $\beta = 0.5$, $\alpha = \dfrac{1-\beta}{L}$, where L is the Lipschitz constant of ∇f, choose $\tilde{w}^0 \in \mathrm{dom}, h$, $\tilde{w}^{-1} = \tilde{w}^0$. The maximal iteration number n_{iter}, and the convergent error ε.
Output: Optimal solution $\tilde{w}^*$ of (10)
Initialize iteration counter $n = 0$
while $n < n_{iter}$ **do**
    $\tilde{w}^{n+1} = (I + \alpha_n \partial g)^{-1}(\tilde{w}^n - \alpha_n \nabla f(\tilde{w}^n) + \beta_n(\tilde{w}^n - \tilde{w}^{n-1}))$
    if $\|\tilde{w}^{n+1} - \tilde{w}^n\|_2 < \varepsilon$ **then**
        break
    end if
    $n = n + 1$
end while
return $\tilde{w}^{n+1}$

Algorithm 2 iPiano for aRSGTSVR

Input: Set step size parameters $\beta = 0.5$, $\alpha = \dfrac{1-\beta}{L}$, where L is the Lipschitz constant of ∇f, choose $\tilde{w}^0 \in \mathrm{dom}, h$, $\tilde{w}^{-1} = \tilde{w}^0$. The maximal iteration number n_{iter}, and the convergent error ε.
Output: Optimal solution $\tilde{w}^*$ of (16)
Initialize iteration counter $n = 0$
while $n < n_{iter}$ **do**
    $\tilde{w}^{n+1} = (I + \alpha_n \partial g)^{-1}(\tilde{w}^n - \alpha_n \nabla f(\tilde{w}^n) + \beta_n(\tilde{w}^n - \tilde{w}^{n-1}))$
    if $\|\tilde{w}^{n+1} - \tilde{w}^n\|_2 < \varepsilon$ **then**
        break
    end if
    $n = n + 1$
end while
return $\tilde{w}^{n+1}$

It is worth noting that the convergence of the proposed iPiano-based algorithm is guaranteed by Theorem 4.8 in the iPiano framework established by Ochs et al. [35]. Specifically, for nonconvex and possibly nonsmooth objective functions, under the condition that the objective satisfies the KurdykaLojasiewicz property and appropriate step size parameters are adopted, it can be verified that the iterative sequence of the iPiano algorithm converges to critical points. Consequently, the algorithm developed has a solid theoretical convergence guaranty.

4. Numerical studies

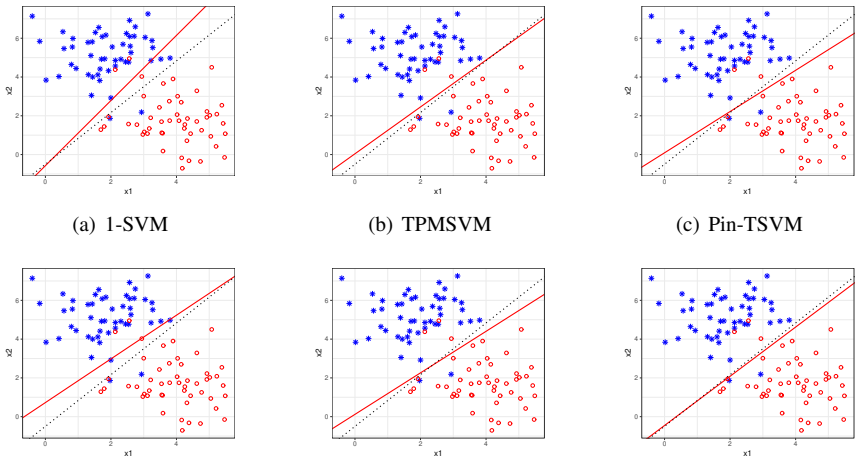
In this section, to further investigate the performance of our proposed algorithm, we design a series of numerical studies to compare aRSGTSVM and aRSGTSVR with some well-known and recent robust methods to verify the performance of our proposed aRSGTSVM and aRSGTSVR.

For the classification problem, we compared the classical 1-SVM [36], TPMSVM [37], Pin-TSVM [31], rhingeSVM [17], and RoBoSS-SVM [20]. The regularization parameter λ in 1-SVM is selected from the candidate set $\Lambda = \{0.01, 0.02, 0.03, \ldots, 0.99\}$. For TPMSVM, we select the values of ν from the set $\{0.1, 0.2, \ldots, 0.8, 0.9\}$. For Pin-TSVM, the parameter ν from set $\{2^i \mid i = -8, -7, \ldots, 7, 8\}$ and τ is tuned in $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. In the rhingeSVM, the η is assumed to be 1. For RoBoSS-SVM, the parameters a and λ are selected from the set $\{1, 2, 3, 4, 5\}$ and $\{0.5, 1, 1.5, 2\}$, respectively. For the aRSGTSVM, the parameters a and λ are the same as those of RoBoSS-SVM, and we take τ in the range $\{0.2, 0.5, 0.8\}$.

For the regression problem, we compared aRSGTSVR with SVR, LASSO, Elastic Net, TSVR [38] and Res-TSVR [39]. In the SVR, the parameter ε is optimized in the range $\{2^i \mid i = -8, -7, \ldots, 7, 8\}$. For TSVR, we set $C_1 = C_2$ and $\varepsilon_1 = \varepsilon_2$. For Res-TSVR, the regularization parameters $C_1 = C_2$ and η in the range of $\{1, 3, \ldots, 16\}$. For the aRSGTSVR, the hyperparameters a, λ, and τ are optimized through a grid search over the discrete sets $\{1, 2, 3, 4, 5\}$, $\{0.5, 1, 1.5, 2\}$, and $\{0.2, 0.5, 0.8\}$, respectively.

For all of the methods mentioned above, we have chosen the values of C and ε from the following sets: $\{2^i \mid i = -8, -7, \ldots, 7, 8\}$. In the nonlinear case, we consider the Gaussian kernel function, i.e. $K(x_1, x_2) = \exp(-\gamma \|x_1 - x_2\|_2^2)$. The parameters γ of the Gaussian kernel function take values in the range $\{2^i \mid i = -8, -7, \ldots, 7, 8\}$. No training-test partition was performed in this study. All models were trained on the entire dataset, and the reported accuracy values are averaged results from five-fold cross-validation, with each fold acting as the validation set sequentially.

For classification problems, using acc (accuracy) as our evaluation metric. For regression problems, we evaluated the performance of the model using RMSE. MAE was used during the simulations to select the optimal parameters. Their specific definitions



(a) 1-SVM (b) TPMSVM (c) Pin-TSVM

(d) rhingeSVM (e) RoBoSS-SVM (f) aRSGTSVM

Figure 2: The red solid lines represent the separating hyperplanes obtained by 1-SVM, TPMSVM, Pin-TSVM, rhingeSVM, RoBoSS-SVM and aRSGTSVM. The Bayes classifier is shown as a black dotted line.

Table 1: The accuracy(acc) and standard deviation (sd) with linear kernel. The bold is the best one.

label noise	**1-SVM** **acc ± sd**	**TPMSVM** **acc ± sd**	**Pin-TSVM** **acc ± sd**	**rhingeSVM** **acc ± sd**	**RoBoSS-SVM** **acc ± sd**	**aRSGTSVM** **acc ± sd**
no noise	0.950 ± 0.035	0.950 ± 0.055	0.950 ± 0.035	0.960 ± 0.055	0.950 ± 0.035	**0.960 ± 0.022**
5%	0.910 ± 0.082	0.910 ± 0.004	0.920 ± 0.057	0.910 ± 0.042	0.910 ± 0.065	**0.920 ± 0.027**
10%	0.850 ± 0.087	0.850 ± 0.004	0.850 ± 0.061	0.850 ± 0.061	0.850 ± 0.094	**0.870 ± 0.057**
20%	0.780 ± 0.104	0.770 ± 0.004	0.770 ± 0.130	0.770 ± 0.168	0.770 ± 0.076	**0.790 ± 0.065**
35%	0.620 ± 0.076	0.630 ± 0.004	0.630 ± 0.091	0.630 ± 0.076	0.630 ± 0.065	**0.660 ± 0.074**

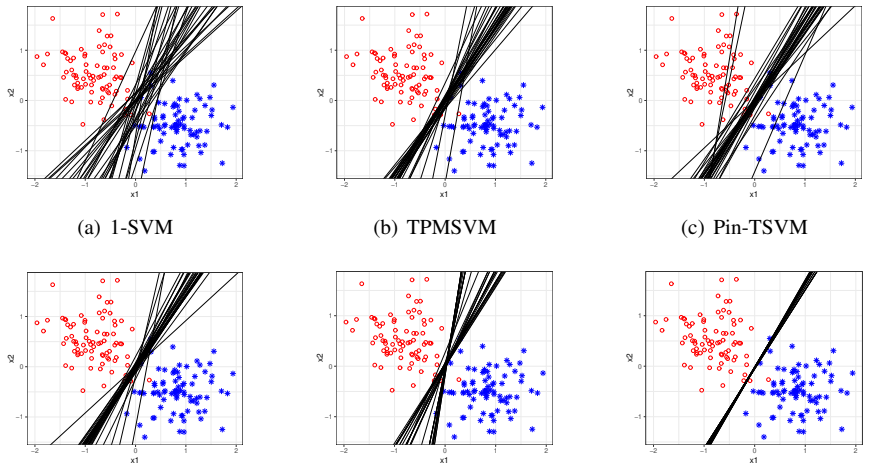
are as follows:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{25}$$

where n represents the number of samples used for testing.

4.1. Artificial datasets for classification

Example 1. To evidence the robustness of aRSGTSVM to label noise, we conducted an experiment using a two-dimensional synthetic dataset containing 100 samples. In this example, the samples were generated from two Gaussian distributions with equal probability: $x_i, i \in \{1, 2, \ldots, 50\} \sim N(u_1, \Sigma_1), x_i, i \in \{51, 52, \ldots, 100\} \sim N(u_2, \Sigma_2)$, where $u_1 = [2, 5]^T, u_2 = [4, 2]^T$ and $\Sigma_1 = \Sigma_2 = \text{diag}(1, 2)$. In this case, the



(a) 1-SVM (b) TPMSVM (c) Pin-TSVM

(d) rhingeSVM (e) RoBoSS-SVM (f) aRSGTSVM

Figure 3: The black solid lines represent the separating hyperplanes obtained by 1-SVM, TPMSVM, Pin-TSVM, rhingeSVM, RoBoSS-SVM and aRSGTSVM.

Bayes classifier is $f_C(x) = 4x_1 - 3x_2 - \frac{3}{2}$. To systematically investigate the impact of label noise on model performance, experiments were conducted under varying noise ratios.

In Figure 2 presents a noise-free condition, in which the black dotted line represents the Bayesian optimal decision boundary and the red solid line represents the decision boundary obtained from each model. Among the six models evaluated, our proposed aRSGTSVM demonstrates the most satisfactory performance, with its decision boundary showing the closest alignment to that of the Bayesian classifier. The complete experimental results are presented in Table 1, which demonstrate the remarkable robustness and noise insensitivity of our proposed aRSGTSVM.

Example 2. To exemplify the stability of SVM under resampling, a total of 160 samples were generated based on Example 1, with the parameters set to $u_1 = [0.8, -0.5]^T$, $u_2 = [-0.8, 0.5]^T$, and $\Sigma_1 = \Sigma_2 = \text{diag}(0.2, 0.2)$. The experiment was repeated 30 times.

As visualized in Figure 3, the proposed aRSGTSVM demonstrates exceptional robustness and stability in constructing decision hyperplanes. Specifically, the hyperplanes of aRSGTSVM form a remarkably narrow and highly concentrated cluster, with almost no variation across different resampling runs, highlighting its strong

Table 2: The accuracy (acc) and standard deviation (sd) in different (n, p). The bold is the best one.

	1-SVM	**TPMSVM**	**Pin-TSVM**	**rhingeSVM**	**RoBoSS-SVM**	**aRSGTSVM**
(a) 0% label noise						
$n = 50, p = 100$	0.900 ± 0.071	0.920 ± 0.130	**0.940 ± 0.055**	0.900 ± 0.071	0.920 ± 0.110	0.940 ± 0.089
$n = 50, p = 150$	0.940 ± 0.055	0.920 ± 0.084	0.940 ± 0.055	0.920 ± 0.084	0.920 ± 0.084	**0.960 ± 0.055**
$n = 100, p = 150$	0.920 ± 0.027	0.930 ± 0.027	0.930 ± 0.076	0.920 ± 0.067	0.930 ± 0.057	**0.970 ± 0.045**
$n = 100, p = 200$	0.920 ± 0.076	0.910 ± 0.055	0.920 ± 0.084	0.910 ± 0.074	0.930 ± 0.045	**0.960 ± 0.042**
(b) 5% label noise						
$n = 50, p = 100$	0.860 ± 0.114	0.880 ± 0.084	0.900 ± 0.122	0.860 ± 0.134	0.880 ± 0.045	**0.920 ± 0.084**
$n = 50, p = 150$	0.940 ± 0.055	0.900 ± 0.071	0.940 ± 0.055	0.920 ± 0.045	0.940 ± 0.089	**0.960 ± 0.055**
$n = 100, p = 150$	0.900 ± 0.035	0.880 ± 0.057	0.890 ± 0.108	0.880 ± 0.084	0.900 ± 0.100	**0.940 ± 0.042**
$n = 100, p = 200$	0.930 ± 0.084	0.920 ± 0.076	0.920 ± 0.057	0.910 ± 0.042	0.930 ± 0.045	**0.950 ± 0.035**
(c) 10% label noise						
$n = 50, p = 100$	0.840 ± 0.134	0.840 ± 0.152	0.860 ± 0.114	0.840 ± 0.114	0.860 ± 0.055	**0.880 ± 0.130**
$n = 50, p = 150$	0.900 ± 0.100	0.880 ± 0.084	0.880 ± 0.084	0.880 ± 0.164	0.900 ± 0.071	**0.920 ± 0.084**
$n = 100, p = 150$	0.830 ± 0.104	0.840 ± 0.114	0.830 ± 0.076	0.810 ± 0.108	0.850 ± 0.087	**0.870 ± 0.097**
$n = 100, p = 200$	0.830 ± 0.097	0.800 ± 0.079	0.810 ± 0.082	0.810 ± 0.124	0.830 ± 0.144	**0.870 ± 0.027**
(d) 20% label noise						
$n = 50, p = 100$	0.720 ± 0.228	0.700 ± 0.122	0.740 ± 0.207	0.720 ± 0.130	0.720 ± 0.179	**0.780 ± 0.110**
$n = 50, p = 150$	0.800 ± 0.071	0.840 ± 0.089	0.780 ± 0.110	0.780 ± 0.110	0.780 ± 0.084	**0.840 ± 0.195**
$n = 100, p = 150$	0.770 ± 0.130	0.770 ± 0.057	0.750 ± 0.112	0.770 ± 0.115	0.770 ± 0.084	**0.800 ± 0.087**
$n = 100, p = 200$	0.770 ± 0.115	0.760 ± 0.134	0.760 ± 0.055	0.740 ± 0.089	0.770 ± 0.104	**0.790 ± 0.089**
(e) 35% label noise						
$n = 50, p = 100$	0.600 ± 0.100	0.600 ± 0.141	0.620 ± 0.084	0.560 ± 0.182	0.660 ± 0.195	**0.680 ± 0.130**
$n = 50, p = 150$	0.700 ± 0.071	0.660 ± 0.167	0.740 ± 0.089	0.660 ± 0.152	0.720 ± 0.148	**0.780 ± 0.205**
$n = 100, p = 150$	0.620 ± 0.057	0.610 ± 0.108	0.630 ± 0.097	0.600 ± 0.127	0.640 ± 0.089	**0.660 ± 0.082**
$n = 100, p = 200$	0.620 ± 0.115	0.600 ± 0.035	0.640 ± 0.055	0.590 ± 0.152	0.620 ± 0.104	**0.670 ± 0.067**

resistance to sample perturbations. While RoBoSS-SVM exhibits improved stability relative to traditional SVM variants, its hyperplanes still show non-negligible dispersion. In sharp contrast, the baseline models, including 1-SVM, TPMSVM, Pin-TSVM and rhingeSVM, generate widely scattered hyperplanes with substantial fluctuations, particularly in the boundary regions between classes. This dispersion reveals their sensitivity to minor changes in the training data, resulting in unstable decision boundaries. Beyond its robustness advantage, aRSGTSVM also achieves the most accurate separation between the two classes (red and blue points), effectively distinguishing between the samples and delivering the best overall classification performance among all evaluated models.

Example 3. In this example, the samples were generated from two multivariate Gaussian distributions with equal probability: $x_i, i \in \{1, 2, \ldots, \frac{n}{2}\} \sim N(u_1, \Sigma_1), x_i, i \in \{\frac{n}{2} + 1, \frac{n}{2} + 2, \ldots, n\} \sim N(u_2, \Sigma_2)$, where $u_1 = (0.1, 0.2, 0.3, 0.4, 0.5, 0, \ldots, 0)^T \in \mathbb{R}^n$, $u_2 = -u_1$ and $\Sigma_1 = \Sigma_2 = (\sigma_{ij})$ is defined with non-zero elements: $\sigma_{ii} = 1$, for $i =$

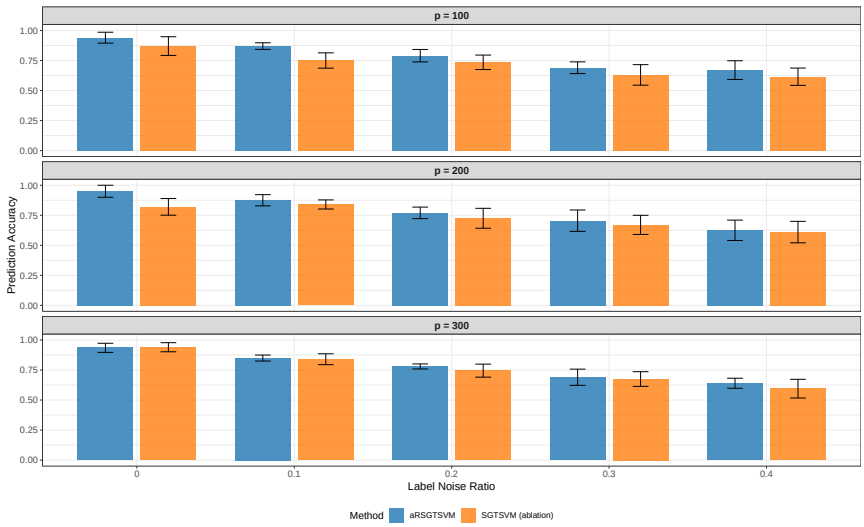


Figure 4: The results of the ablation experiments under different ratios of label noise and number of features.

$1, 2, \ldots, n$ and $\sigma_{ij} = -0.2$, for $1 \leq i \neq j \leq 5$.

As shown in Table 2, we systematically compared the classification performance of the proposed aRSGTSVM with several state-of-the-art SVM-based methods across different combinations of sample size n, feature dimension p, and varying label noise levels ranging from 0% to 35%. The results, measured by mean classification accuracy and standard deviation, consistently demonstrate that aRSGTSVM achieves the highest or tied-for-highest accuracy in nearly all experimental settings, including high-dimensional scenarios where $p > n$, while maintaining stable performance across repeated trials. As label noise increases, aRSGTSVM consistently outperforms competing methods, showing its superiority in both clean high-dimensional environments and noisy conditions. This comprehensive comparison confirms that the proposed aRSGTSVM exhibits stronger robustness against label noise and better adaptability to high-dimensional data compared with baseline models, validating the effectiveness of its asymmetric regularized twin SVM framework.

Example 4. To further verify the effectiveness of the proposed asymmetric RoBoSS loss function, we conduct ablation experiments. Based on the setting of Example 3, we fix the sample size at $n = 200$, set the feature dimension $p \in \{100, 200, 300\}$

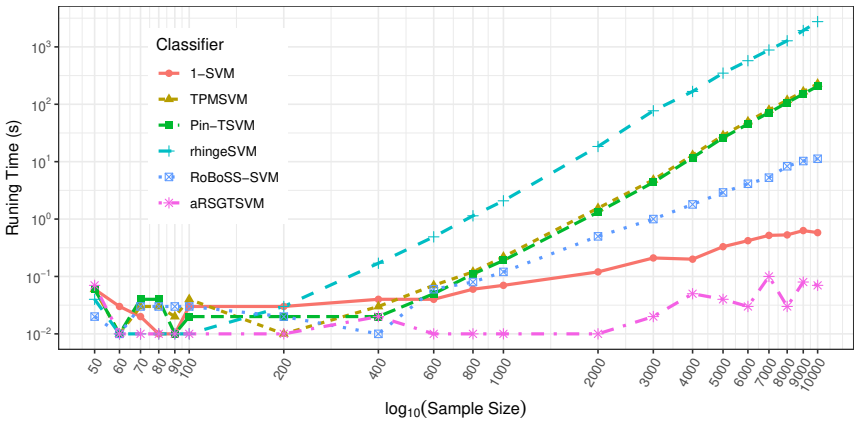


Figure 5: The one-run CPU time cost of 1-SVM, TPMSVM, Pin-TSVM, rhingeSVM, RoBoSS-SVM and aRSGTSVM under different sample sizes.

and the label noise ratio to $\{0\%, 10\%, 20\%, 30\%, 40\%\}$. The experimental results are presented in Figure 4.

As shown in Figure 4, under different feature dimensions ($p = 100, 200, 300$) and varying label noise ratios (0% to 40%), the proposed aRSGTSVM consistently achieves higher prediction accuracy than the ablation method SGTSVM, which removes the asymmetric robust loss. As the noise ratio increases from 0% to 40%, the prediction accuracy of both methods decreases. However, aRSGTSVM exhibits a much smaller decline, demonstrating superior robustness to label noise. Moreover, as the feature dimension increases from 100 to 300, aRSGTSVM maintains stable performance, whereas SGTSVM shows a more pronounced drop in accuracy. These results validate the effectiveness and robustness of the proposed asymmetric RoBoSS loss function in scenarios with label noise and high-dimensional features.

Example 5. In this example, we investigate the CPU time cost of our proposed algorithm (Algorithm 1) with varying numbers of samples and features, respectively. Based on the setting of Example 1, we fix the feature dimension at $p = 2$, set the sample size n ranging from 50 to 10000 with a label noise ratio of 15%, and set all parameters to 1 except for $\tau = 0.5$. The results are presented in Figure 5. Based on the setting of Example 3, We further conduct experiments with a fixed sample size $n = 100$ and feature dimension p ranging from 50 to 5000, under the same noise ratio and parameter

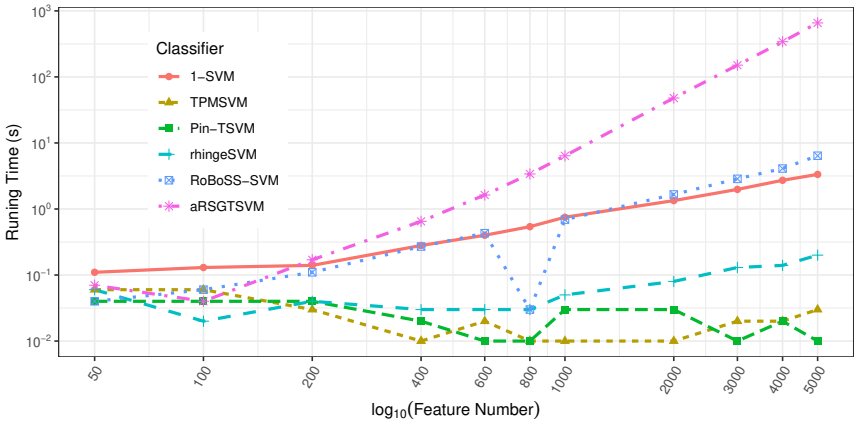
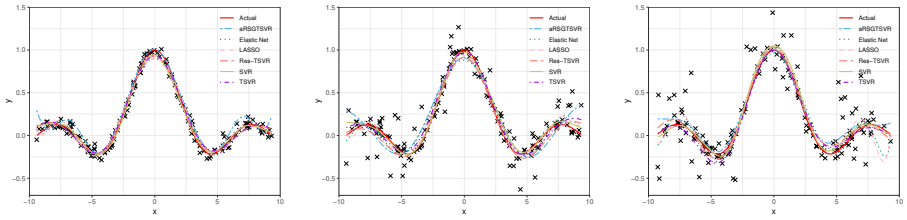


Figure 6: The one-run CPU time cost of 1-SVM, TPMSVM, Pin-TSVM, rhingeSVM, RoBoSS-SVM and aRSGTSVM under different feature numbers.

settings, as shown in Figure 6.

Figure 5 presents the CPU running time of various classifiers under different sample sizes. As the sample size increases from 50 to 10000, the running time of most compared algorithms increases significantly. Among them, rhingeSVM and Pin-TSVM show the fastest growth in time complexity, with their time consumption far exceeding other models in large-sample scenarios. In contrast, the proposed aRSGTSVM maintains an extremely low running time throughout. Even when the sample size reaches 10000, its time consumption remains at a low level, only slightly higher than 1-SVM and much lower than other compared models, demonstrating excellent scalability to large sample sizes.

Figure 6 illustrates the single-run CPU time (log scale) of different algorithms as the feature dimension increases from 50 to 5,000, with a fixed sample size of $n = 100$. For the proposed aRSGTSVM, the running time grows slowly when the number of features is below 200, but increases sharply thereafter, becoming the highest among all compared methods at 5,000 dimensions. In contrast, Pin-TSVM and TPMSVM maintain consistently low running times, while 1-SVM and RoBoSS-SVM exhibit an approximately linear increase with a much lower growth rate than aRSGTSVM. In summary, aRSGTSVM is sensitive to feature dimensionality, incurring high computational costs in high-dimensional scenarios, though its running time remains acceptable



(a) No noise (b) Uniform noise (c) Gaussian noise

Figure 7: Predictions of SVR, TSVR, Res-TSVR and aRSGTSVR on Sinc function with different noises.

in low-to-medium dimensions.

4.2. Artificial datasets for regression

Example 1. In this example, we designed a two-dimensional case to demonstrate that aRSGTSVR is robust under different types of noise, namely uniform noise and Gaussian noise. We generate 160 samples as follows:

$$y = \mathrm{sinc}(x_i) = \frac{\sin x_i}{x_i} + e_i, \quad x_i \sim U[-3\pi, 3\pi]. \tag{26}$$

Noise is added based on (26), which is defined as follows:

$$\text{Type A: } e_i \sim U[-0.4, 0.4], \text{Type B: } e_i \sim N(0, 0.2^2).$$

We plotted the fitting curve of each model in Figure 7. Since it is not intuitively obvious which model fits the true curve better, we present the corresponding root mean square error and standard deviation obtained with the nonlinear kernel setting in Table 3. The results consistently show that the proposed aRSGTSVR achieves the best overall performance in terms of RMSE across different noise conditions, including noiseless, uniform noise, and Gaussian noise scenarios, while maintaining a relatively low standard deviation. Compared with competing methods such as SVR, LASSO, Elastic Net, TSVR, and Res-TSVR, aRSGTSVR exhibits more stable and superior regression performance, which confirms that our method is more robust than other comparative models under different types of noise when handling nonlinear regression tasks.

Table 3: The RMSE and standard deviation (sd) with nonlinear kernel. The bold is the best one.

	SVR	**LASSO**	**Elastic Net**	**TSVR**	**Res-TSVR**	**aRSGTSVR**
No noise	0.080 ± 0.013	0.074 ± 0.012	0.075 ± 0.011	0.075 ± 0.030	0.126 ± 0.115	**0.071 ± 0.010**
Uniform noise	0.128 ± 0.026	0.129 ± 0.014	0.130 ± 0.015	0.129 ± 0.021	0.128 ± 0.022	**0.118 ± 0.017**
Gaussian noise	0.267 ± 0.059	0.270 ± 0.062	0.265 ± 0.060	0.265 ± 0.055	0.264 ± 0.057	**0.260 ± 0.073**

Table 4: The RMSE and standard deviation (sd) in different (n, p). The bold is the best one.

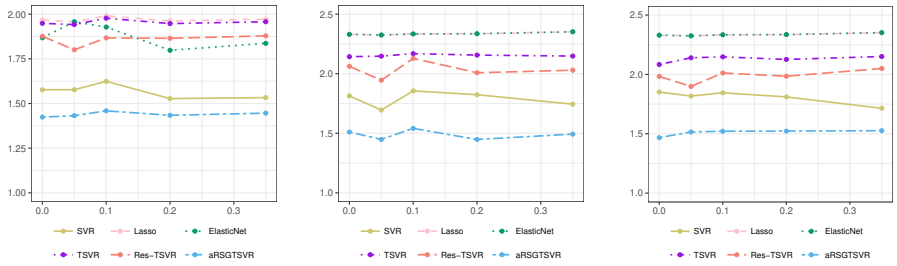
(a) 0% label noise						
	SVR	**LASSO**	**Elastic Net**	**TSVR**	**Res-TSVR**	**aRSGTSVR**
$n = 50, p = 30$	1.570 ± 1.342	1.968 ± 1.988	1.819 ± 1.838	1.949 ± 1.254	1.861 ± 0.892	**1.438 ± 0.675**
$n = 50, p = 50$	1.712 ± 1.902	2.331 ± 2.354	2.331 ± 2.354	2.105 ± 1.998	1.976 ± 0.907	**1.481 ± 1.041**
$n = 50, p = 100$	1.811 ± 1.900	2.331 ± 2.354	2.331 ± 2.354	2.140 ± 0.884	2.041 ± 2.532	**1.501 ± 1.135**
(b) 5% label noise						
$n = 50, p = 30$	1.574 ± 1.341	1.958 ± 1.977	1.958 ± 1.977	1.942 ± 1.469	1.862 ± 0.770	**1.406 ± 1.101**
$n = 50, p = 50$	1.802 ± 1.607	2.325 ± 2.349	2.325 ± 2.349	2.097 ± 1.618	1.904 ± 1.142	**1.461 ± 1.543**
$n = 50, p = 100$	1.773 ± 1.478	2.325 ± 2.349	2.325 ± 2.349	2.157 ± 2.204	1.986 ± 1.350	**1.480 ± 1.438**
(c) 10% label noise						
$n = 50, p = 30$	1.632 ± 0.574	1.990 ± 2.010	1.990 ± 2.010	1.978 ± 1.155	1.892 ± 0.922	**1.444 ± 0.717**
$n = 50, p = 50$	1.835 ± 0.774	2.334 ± 2.358	2.334 ± 2.358	2.178 ± 2.055	2.091 ± 0.784	**1.487 ± 1.290**
$n = 50, p = 100$	1.878 ± 1.115	2.334 ± 2.358	2.334 ± 2.358	2.165 ± 1.645	2.120 ± 1.348	**1.483 ± 1.279**
(d) 20% label noise						
$n = 50, p = 30$	1.590 ± 0.477	1.962 ± 1.982	1.825 ± 1.843	1.946 ± 1.596	1.709 ± 1.477	**1.435 ± 0.994**
$n = 50, p = 50$	1.815 ± 1.876	2.336 ± 2.360	2.336 ± 2.360	2.150 ± 1.257	1.941 ± 1.999	**1.485 ± 1.315**
$n = 50, p = 100$	1.832 ± 2.459	2.336 ± 2.360	2.336 ± 2.360	2.162 ± 1.702	1.924 ± 1.491	**1.535 ± 1.984**
(e) 35% label noise						
$n = 50, p = 30$	1.590 ± 0.360	1.972 ± 1.992	1.823 ± 1.841	1.896 ± 1.044	1.858 ± 1.041	**1.400 ± 0.959**
$n = 50, p = 50$	1.767 ± 1.566	2.352 ± 2.376	2.352 ± 2.376	2.169 ± 1.798	1.939 ± 1.758	**1.528 ± 1.302**
$n = 50, p = 100$	1.787 ± 1.583	2.352 ± 2.376	2.352 ± 2.376	2.157 ± 2.144	2.002 ± 1.248	**1.519 ± 1.682**

Example 2. To illustrate that aRSGTSVR exhibits excellent variable selection capability under different noise ratios for high-dimensional data, in this example, the samples are generated from the following multivariate Gaussian distribution: $x_i, i \in \{1, 2, \ldots, n\} \sim N(u, \Sigma)$, where $u = (0.1, 0.2, 0.3, 0.4, 0.5, 0, \ldots, 0)^T \in \mathbb{R}^n$ and $\Sigma = (\sigma_{ij})$ is defined with non-zero elements: $\sigma_{ii} = 1$, for $i = 1, 2, \ldots, n$ and $\sigma_{ij} = -0.2$, for $1 \leq i \neq j \leq 5$.

The response variable y for each sample is constructed via the function:

$$y = \sin(3x_1) + x_2^2 + \exp(-|x_3|) + \log(|x_4| + 1) + x_1 x_5. \tag{27}$$

Table 4 presents the RMSE and standard deviation results of all compared methods across different combinations of sample size n, feature dimension p, and label noise levels ranging from 0% to 35%, while Figure 8 further visualizes the perfor-



(a) $n = 50, p = 30$ (b) $n = 50, p = 50$ (c) $n = 50, p = 100$

Figure 8: Predictions of SVR, LASSO, Elastic Net, TSVR, Res-TSVR and aRSGTSVR on Sinc function with different types of noises.

mance trends of each method under varying noise conditions for different dimensional settings. As shown in both the table and figure, the proposed aRSGTSVR consistently achieves the lowest RMSE values across all experimental scenarios, including low-dimensional, boundary high-dimensional, and classical high-dimensional cases, while maintaining stable performance with relatively small standard deviations. Compared with baseline models such as SVR, LASSO, Elastic Net, TSVR, and Res-TSVR, aRSGTSVR exhibits superior regression accuracy and stronger robustness against increasing label noise, confirming its combined advantages in prediction accuracy and stability for both low- and high-dimensional nonlinear regression tasks.

4.3. UCI datasets for classification

4.3.1. Experimental results

To assess the generalization performance of the models, we conducted tests on a lot of UCI datasets (see Table 5). Furthermore, to examine the robustness of the SVMs to outliers, we introduced artificial noise by randomly selecting 15% and 35% of the training samples and swapping their labels.

Table 6 presents the performance comparison of various SVM models using the initial data. In terms of average prediction accuracy, our proposed aRSGTSVM achieves the most optimal results, followed by RoBoSS-SVM, while rhingeSVM also demonstrates strong competitiveness. Although 1-SVM performs well only on certain datasets, all models can effectively predict data labels. TPMSVM and Pin-TSVM show the

Table 5: The information of seleced UCI datasets.

Dataset	#Samples	#Features
acoustic	400	50
amphibians	189	21
autism	702	15
bads	1372	4
dccc	30000	23
diabetes	520	16
gait	47	321
garments	1197	7
heartfailure	299	12
lsvt	126	310
messidor	1151	19
raisin	900	7
shillbidding	6321	9
tripadvisor	980	9
vertebral	310	6
waveform	5000	40
wp	4746	15

relatively poorest performance. Overall, the vast majority of models are capable of effectively classifying the data.

As shown in Table 7, the aRSGTSVM model still achieves the optimal performance. In comparison, the competitive advantages of rhingeSVM, RoBoSS-SVM, and 1-SVM models gradually diminish, while TPMSVM shows an improving trend. Pin-TSVM consistently demonstrates relatively poor performance. From Table 8, it can be observed that aRSGTSVM performs best on all UCI datasets, and as the noise ratio increases, the superiority of aRSGTSVM becomes more pronounced, indicating that our model exhibits strong robustness to label noise.

4.3.2. Comparisons by statistical test

To further validate the effectiveness of our proposed aRSGTSVM model, we compare it with other SVM models on multiple datasets using the Friedman test and the corresponding Nemenyi post-hoc test[40]. The null hypothesis of the Friedman test is that there is no significant difference among all models. If the null hypothesis is rejected, the Nemenyi test is conducted. The Friedman statistic is defined as:

$$F_F = \frac{(D_n - 1)\chi_F^2}{D_n(C_k - 1) - \chi_F^2},$$

Table 6: The results of UCI datasets with 0% label noise. The bold is the best one.

	1-SVM	TPMSVM	Pin-TSVM	rhingeSVM	RoBoSS-SVM	aRSGTSVM
acoustic	0.755±0.084	0.825±0.062	0.767±0.059	0.868±0.044	0.853±0.046	**0.873±0.022**
amphibians	0.704±0.080	0.746±0.090	0.714±0.115	0.793±0.062	0.809±0.075	**0.814±0.083**
autism	**1.000±0.000**	0.967±0.030	0.970±0.038	**1.000±0.000**	**1.000±0.000**	**1.000±0.000**
bads	0.981±0.017	0.985±0.012	0.961±0.026	0.987±0.009	0.984±0.013	**0.988±0.013**
dccc	0.753±0.064	0.821±0.053	0.812±0.049	0.803±0.054	0.832±0.059	**0.836±0.031**
diabetes	0.917±0.036	0.904±0.037	0.892±0.030	0.937±0.041	**0.940±0.039**	0.910±0.026
gait	0.980±0.063	0.915±0.145	0.915±0.145	0.980±0.063	0.940±0.135	**1.000±0.000**
garments	0.999±0.003	0.998±0.004	0.993±0.008	**1.000±0.000**	**1.000±0.000**	0.999±0.003
heartfailure	0.762±0.071	0.836±0.058	0.843±0.052	0.833±0.079	0.836±0.060	**0.859±0.063**
lsvt	**0.985±0.032**	0.760±0.123	0.824±0.094	0.953±0.075	0.904±0.123	0.912±0.090
messidor	0.600±0.051	0.699±0.042	0.601±0.071	**0.751±0.036**	0.673±0.060	**0.751±0.034**
raisin	0.788±0.034	0.865±0.028	0.862±0.031	0.871±0.032	**0.875±0.027**	0.869±0.027
shillbidding	0.982±0.012	0.984±0.008	0.979±0.010	0.984±0.011	0.982±0.006	**0.988±0.009**
tripadvisor	0.644±0.027	0.747±0.020	0.753±0.019	0.733±0.026	0.759±0.033	**0.761±0.034**
vertebral	0.813±0.052	0.803±0.051	0.806±0.059	0.855±0.061	**0.858±0.046**	0.839±0.061
waveform	0.832±0.047	0.866±0.034	0.836±0.028	0.877±0.021	0.881±0.031	**0.887±0.025**
wp	0.809±0.044	0.853±0.030	0.853±0.028	0.856±0.031	0.857±0.029	**0.861±0.032**

Table 7: The results of UCI datasets with 15% label noise. The bold is the best one.

	1-SVM	TPMSVM	Pin-TSVM	rhingeSVM	RoBoSS-SVM	aRSGTSVM
acoustic	0.670±0.079	0.760±0.079	0.755±0.077	0.767±0.060	0.780±0.088	**0.850±0.057**
amphibians	0.688±0.075	0.730±0.067	0.730±0.114	0.778±0.077	0.740±0.078	**0.784±0.106**
autism	0.829±0.127	0.959±0.025	0.953±0.025	0.949±0.024	0.963±0.026	**0.992±0.015**
bads	0.817±0.032	0.970±0.027	0.955±0.025	0.973±0.024	0.977±0.016	**0.978±0.020**
dccc	0.628±0.063	**0.820±0.041**	0.807±0.040	0.814±0.039	0.817±0.041	0.810±0.041
diabetes	0.691±0.084	0.908±0.031	0.902±0.044	0.869±0.039	**0.917±0.016**	0.904±0.035
gait	0.850±0.141	0.875±0.144	0.855±0.138	0.875±0.144	0.855±0.138	**0.960±0.084**
garments	0.818±0.074	0.994±0.008	0.992±0.009	0.980±0.011	**0.999±0.003**	0.998±0.004
heartfailure	0.659±0.076	0.826±0.075	0.826±0.075	0.823±0.059	0.830±0.069	**0.856±0.074**
lsvt	0.849±0.060	0.754±0.136	0.693±0.120	0.848±0.063	0.819±0.120	**0.921±0.074**
messidor	0.561±0.042	0.698±0.045	0.642±0.035	0.699±0.040	0.646±0.053	**0.731±0.051**
raisin	0.735±0.183	0.859±0.035	0.854±0.032	0.859±0.041	0.863±0.035	**0.871±0.033**
shillbidding	0.903±0.058	0.973±0.012	0.968±0.022	**0.978±0.011**	**0.978±0.010**	**0.978±0.013**
tripadvisor	0.601±0.035	**0.752±0.035**	0.747±0.035	0.731±0.041	0.750±0.042	0.739±0.044
vertebral	0.687±0.087	0.806±0.086	0.742±0.083	0.826±0.072	**0.858±0.059**	**0.858±0.055**
waveform	0.707±0.050	0.815±0.028	0.828±0.032	0.843±0.030	0.831±0.034	**0.860±0.032**
wp	0.669±0.043	0.827±0.031	0.818±0.026	0.824±0.030	0.827±0.026	**0.831±0.033**

where D_n is the number of datasets, and C_k is the number of models,

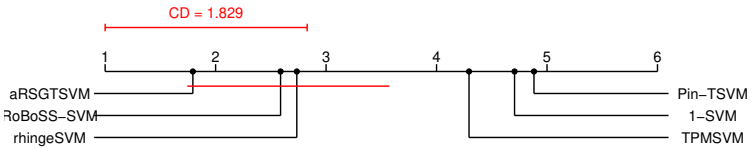
$$\chi_F^2 = \frac{12D_n}{C_k(C_k+1)}\left(\sum_{i=1}^{C_k} R_i^2 - \frac{C_k(C_k+1)^2}{4}\right),$$

where R_i is the average rank of the i-th model (calculated from the results in Table 6-8).

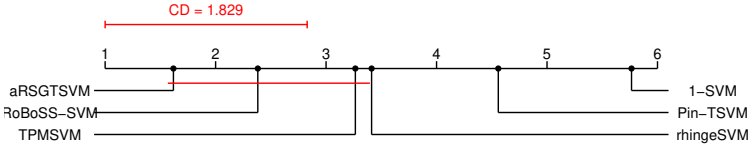
Given $D_n = 17$ and $C_k = 6$, then for a noise ratio of 0%, $F_F = 14.507$. For 15%, $F_F = 27.788$. For 35%, $F_F = 25.107$. In our experiment, F_F follows an $F(5, 80)$ distribution. At a significance level of 0.05, the critical value is $F_{0.05}(5, 80) = 3.329$.

Table 8: The results of UCI datasets with 35% label noise. The bold is the best one.

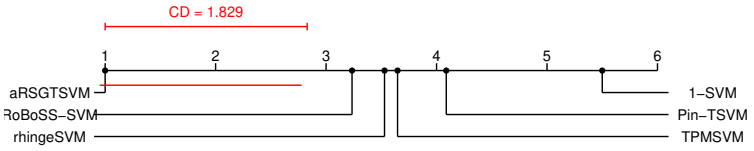
	1-SVM	**TPMSVM**	**Pin-TSVM**	**rhingeSVM**	**RoBoSS-SVM**	**aRSGTSVM**
acoustic	0.585±0.110	0.605±0.112	0.595±0.112	0.647±0.088	0.642±0.093	**0.853±0.053**
amphibians	0.529±0.153	0.546±0.152	0.593±0.098	0.609±0.128	0.556±0.113	**0.793±0.069**
autism	0.619±0.294	0.882±0.050	0.929±0.033	0.895±0.057	0.900±0.056	**0.993±0.022**
bads	0.620±0.054	0.972±0.014	0.918±0.038	0.965±0.018	0.969±0.014	**0.981±0.011**
dccc	0.570±0.113	0.778±0.056	0.549±0.110	0.787±0.047	0.779±0.043	**0.799±0.050**
diabetes	0.610±0.171	0.810±0.066	0.835±0.052	0.821±0.048	0.835±0.047	**0.902±0.032**
gait	0.755±0.227	0.640±0.251	0.685±0.221	0.680±0.258	0.620±0.253	**0.980±0.063**
garments	0.705±0.348	0.973±0.015	0.985±0.013	0.969±0.023	0.989±0.011	**0.999±0.003**
heartfailure	0.519±0.156	0.736±0.139	0.690±0.137	0.719±0.110	0.733±0.108	**0.853±0.039**
lsvt	0.666±0.158	0.608±0.191	0.586±0.193	0.668±0.164	0.666±0.148	**0.912±0.081**
messidor	0.554±0.045	0.617±0.052	0.557±0.063	0.590±0.071	0.602±0.065	**0.717±0.035**
raisin	0.615±0.295	0.848±0.038	0.846±0.039	0.836±0.035	0.851±0.040	**0.868±0.027**
shillbidding	0.632±0.073	0.973±0.018	0.975±0.016	0.970±0.016	0.971±0.017	**0.981±0.014**
tripadvisor	0.535±0.046	0.668±0.084	0.602±0.179	0.728±0.058	0.714±0.071	**0.742±0.053**
vertebral	0.635±0.171	0.716±0.087	0.603±0.143	0.707±0.062	0.687±0.063	**0.855±0.060**
waveform	0.612±0.065	0.802±0.051	0.804±0.047	0.779±0.051	0.793±0.060	**0.877±0.021**
wp	0.566±0.088	0.798±0.033	0.786±0.061	0.804±0.051	0.816±0.034	**0.832±0.034**



(a) 0% label noise



(b) 15% label noise



(c) 35% label noise

Figure 9: The average ranks of six classifiers on the UCI datasets with different label noise.

Since $F_F > 3.329$, the null hypothesis of the Friedman test should be rejected, and the Nemenyi test is conducted next.

If there is a significant difference between two models, the difference in their average ranks must be at least the critical difference, defined as:

$$CD = q_{0.05}\sqrt{\frac{C_k(C_k+1)}{6D_n}}.$$

Given that $D_n = 17$, $C_k = 6$, and $q_{0.05}(6) = 2.85$, we obtain $CD = 1.777$.

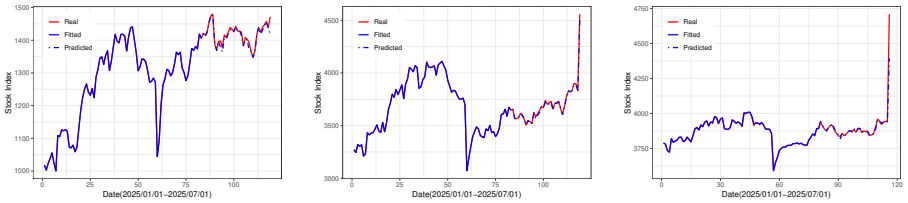
Figure 9 visualizes the average ranking comparison of six Support Vector Regression (SVR) models. According to the Critical Difference (CD) diagram analysis criterion, when the distance between two models on the horizontal axis exceeds the CD value (red solid line), it indicates a statistically significant difference between them. In the subfigure 9(a), aRSGTSVM, rhingeSVM, and RoBoSS-SVM are grouped together, while Pin-TSVM, 1-SVM, and TPMSVM form another group, with significant performance differences between these two groups. In the subfigure 9(b), the grouping changes to aRSGTSVM, RoBoSS-SVM, and TPMSVM as one group, and 1-SVM, Pin-TSVM, and rhingeSVM as the other group, maintaining significant inter-group differences. In the subfigure 9(c), aRSGTSVM stands alone as one group, while the remaining models (RoBoSS-SVM, rhingeSVM, 1-SVM, Pin-TSVM, and TPMSVM) form another group, with equally significant performance differences between groups. It can be observed that aRSGTSVM significantly outperforms other comparative models, and its advantage becomes more pronounced as the noise ratio increases, fully demonstrating the model's performance stability across different label noise scenarios.

4.4. Index tracking for regression

In this section, to assess the models out-of-sample performance, we employ six index tracking datasets spanning from January 1, 2025, to July 1, 2025, for experimentation. The dataset is partitioned such that the first 70% (rounded down if not an integer) is used as the training set, with the remainder serving as the test set. During training, the Mean Absolute Error (MAE) is utilized to select the optimal parameters, while the model's performance is evaluated based on the annual tracking error. The annual tracking error is defined as:

Table 9: TrackingError$_{Year}$ for different datasets. The bold is the best one.

Index	SVR	LASSO	Elastic Net	TSVR	Res-TSVR	aRSGTSVR
bz50	0.138	0.126	0.133	0.120	0.135	**0.117**
cy200	0.059	0.134	0.078	0.045	0.045	**0.040**
hs300	0.277	0.277	0.380	0.122	0.138	**0.117**
xf100	0.638	0.328	0.250	0.368	0.925	**0.226**
ys50	0.191	0.065	0.068	0.128	0.124	**0.057**
zz500	0.297	0.271	0.185	0.162	0.109	**0.105**



(a) bz50 (b) cy200 (c) hs300

(d) xf100 (e) ys50 (f) zz500

Figure 10: Index tracking plots for Stock Indices. The red solid lines are the true values, the blue solid lines are the fitted values on the training set, and the blue dashed lines are the predicted values on the test set.

$$\text{TrackingError}_{\text{year}} = \sqrt{252} \cdot \sqrt{\frac{\sum_{t=1}^{T} (err_t - \overline{err})^2}{T-1}},$$

where $err_t = y_t - \hat{y}_t$ for $t = 1, 2, \cdots, T$, y_t denotes the return of an index at time t, $\hat{y}_t$ is the estimated value of y_t, and $\overline{err}$ represents the mean of err_t over $t = 1, 2, \cdots, T$.

To mitigate spurious regression and enable the model to focus on replicating relative price movements, converting raw price data to returns is crucial for index tracking. For a raw price series P_t, the return at time t is calculated as:

$$r_t = \frac{P_t}{P_{t-1}} - 1, \quad t = 1, 2, \cdots, T.$$

Similarly, the index value y_t at time t is converted to its corresponding return, which

we continue to denote as y_t for consistency.

The experimental results are presented in Table 9. As evidenced by the annual tracking errors, the proposed aRSGTSVR achieves the best performance across all six index tracking datasets, demonstrating strong generalization capability of the model. As illustrated in Figure 10, the fitted and predicted values of the aRSGTSVR model closely align with the true values. This visually confirms the model's superior fitting performance and robust generalization ability.

5. Conclusion

This paper designs a novel asymmetric loss function L_{aR} based on the RoBoSS loss, which is integrated with ENNHSVM to establish aRSGTSVM for classification and aRSGTSVR for regression tasks. To tackle the challenging optimization issue arising from the combination of a nonconvex loss L_{aR} and a sparse penalty l_1 in the objective function, the iPiano algorithm is adopted to achieve efficient and stable optimization. Experimental results in synthetic datasets demonstrate that aRSGTSVM maintains strong robustness against label noise, delivers stable performance under resampling, achieves satisfactory results in high-dimensional scenarios, and presents competitive computational efficiency. Meanwhile, aRSGTSVR also exhibits prominent noise resistance and effective high-dimensional variable selection capability. In real-world datasets, both proposed models acquire favorable generalization performance. The core contribution of this study lies in that the designed asymmetric loss naturally balances model robustness and sparsity, which provides a reliable and superior alternative to conventional symmetric loss functions for practical scenarios suffering from label noise, high-dimensional features, and resampling instability.

It is noteworthy that although this study employs a cross-validation strategy for model parameter optimization, this method can only detect the presence or absence of overfitting rather than alleviate it. Alleviating overfitting requires strategies such as regularization. Furthermore, cross-validation necessitates repeated data partitioning and model retraining, resulting in substantial computational overhead. This issue becomes particularly pronounced in high-dimensional data or complex model scenarios. Given

that information criteria (e.g., AIC, BIC) can construct objective functions by integrating model goodness of fit and complexity without requiring repeated data partitioning and retraining, they significantly enhance parameter tuning efficiency while balancing model generalization performance. Therefore, exploring the optimal information criterion tailored to the characteristics of our model represents a highly valuable direction for future research. Additionally, distributed or parallel algorithms can be employed to further improve the computational efficiency of the model.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant No. 12401664), the Natural Science Foundation of Chongqing Municipality (Grant No. CSTB2024NSCQ-MSX0855), and the Science and Technology Research Program of Chongqing Municipal Education Commission (Grant No. KJQN202400514).

References

[1] Jayadeva, R. Khemchandani, S. Chandra, Twin support vector machines for pattern classification, IEEE Transactions on Pattern Analysis and Machine Intelligence 29 (2007) 905–910.

[2] X. Chen, J. Yang, Q. Ye, J. Liang, Recursive projection twin support vector machine via within-class variance minimization, Pattern Recognition 44 (2011) 2643–2655.

[3] K. Qi, H. Yang, Elastic net nonparallel hyperplane support vector machine and its geometrical rationality, IEEE Transactions on Neural Networks & Learning Systems 33 (2022) 7199–7209.

[4] Z. Liang, S. Ding, Fuzzy twin support vector machines with distribution inputs, IEEE Transactions on Fuzzy Systems 32 (2024) 240–254.

[5] A. Quadir, M. Sajid, M. Tanveer, Granular ball twin support vector machine, IEEE Transactions on Neural Networks & Learning Systems 36 (2025) 12444–12453.

[6] Y. Shao, C. Zhang, X. Wang, N. Deng, Improvements on twin support vector machines, IEEE Transactions on Neural Networks 22 (2011) 962–968.

[7] Y. Wu, Y. Liu, Robust truncated hinge loss support vector machines, Journal of the American Statistical Association 102 (2007) 974–983.

[8] H. Wang, Z. Zhu, Y. Shao, Fast support vector machine with low-computational complexity for large-scale classification, IEEE Transactions on Systems, Man, & Cybernetics: Systems 54 (2024) 4151–4163.

[9] X. Huang, L. Shi, J. Suykens, Support vector machine classifier with pinball loss, IEEE Transactions on Pattern Analysis & Machine Intelligence 36 (2014) 984–997.

[10] J. Zhu, S. Rosset, T. Hastie, R. Tibshirani, 1-norm support vector machines, in: Advances in Neural Information Processing Systems, 2004, pp. 1–8.

[11] L. Wang, J. Zhu, H. Zou, The doubly regularized support vector machine, Statistica Sinica (2006) 589–615.

[12] N. Krause, Y. Singer, Leveraging the margin more carefully, in: Proceedings of the Twenty-First International Conference on Machine Learning, 2004, p. 63.

[13] D. Liu, Y. Shi, Y. Tian, Ramp loss nonparallel support vector machine for pattern classification, Knowledge-Based Systems 85 (2015) 224–233.

[14] C. Wang, Q. Ye, P. Luo, N. Ye, L. Fu, Robust capped l1-norm twin support vector machine, Neural Networks 114 (2019) 47–59.

[15] W. Liu, P. Pokharel, J. Principe, Correntropy: Properties and applications in non-gaussian signal processing, IEEE Transactions on Signal Processing 55 (2007) 5286–5298.

[16] A. Singh, R. Pokharel, J. Principe, The c-loss function for pattern classification, Pattern Recognition 47 (2014) 441–453.

[17] G. Xu, Z. Cao, B. Hu, J. Principe, Robust support vector machines based on the rescaled hinge loss function, Pattern Recognition 63 (2017) 139–148.

[18] G. Xu, B. Hu, J. Principe, Robust c-loss kernel classifiers, IEEE Transactions on Neural Networks & Learning Systems 29 (2018) 510–529.

[19] J. Ma, L. Yang, Q. Sun, Adaptive robust learning framework for twin support vector machine classification, Knowledge-Based Systems 211 (2021) 106536.

[20] M. Akhtar, M. Tanveer, M. Arshad, Roboss: A robust, bounded, sparse, and smooth loss function for supervised learning, IEEE Transactions on Pattern Analysis & Machine Intelligence 47 (2025) 149–160.

[21] X. Huang, L. Shi, J. Suykens, Asymmetric least squares support vector machine classifiers, Computational Statistics & Data Analysis 70 (2014) 395–405.

[22] X. Shen, L. Niu, Z. Qi, Y. Tian, Support vector machine classifier with truncated pinball loss, Pattern Recognition 68 (2017) 199–210.

[23] L. Yang, H. Dong, Robust support vector machine with generalized quantile loss for classification and regression, Applied Soft Computing 81 (2019) 105483.

[24] M. He, F. He, L. Shi, X. Huang, J. Suykens, Learning with asymmetric kernels: Least squares and feature interpretation, IEEE Transactions on Pattern Analysis & Machine Intelligence 45 (2023) 10044–10054.

[25] K. Ikeda, N. Murata, Geometrical properties of nu support vector machines with different norms, Neural Computation 17 (2005) 2508–2529.

[26] H. Zou, M. Yuan, The F_∞-norm support vector machine, Statistica Sinica (2008) 379–398.

[27] S. Gao, Q. Ye, N. Ye, 1-norm least squares twin support vector machines, Neurocomputing 74 (2011) 3590–3597.

[28] H. Moosaei, M. Hladík, Sparse solution of least-squares twin multi-class support vector machine using ℓ_0 and ℓ_p-norm for classification and feature selection, Neural Networks 166 (2023) 471–486.

[29] X. Xie, F. Sun, J. Qian, L. Guo, R. Zhang, X. Ye, Z. Wang, Laplacian l_p norm least squares twin support vector machine, Pattern Recognition 136 (2023) 109192.

[30] Y.-H. Shao, W.-J. Chen, N.-Y. Deng, Nonparallel hyperplane support vector machine for binary classification problems, Information Sciences 263 (2014) 22–35.

[31] Y. Xu, Z. Yang, X. Pan, A novel twin support-vector machine with pinball loss, IEEE Transactions on Neural Networks & Learning Systems 28 (2017) 359–370.

[32] F. R. Hampel, Contributions to the theory of robust estimation, University of California, Berkeley, 1968.

[33] M. Akhtar, M. Tanveer, M. Arshad, Robots: A robust bounded twin svm based on roboss loss function, Pattern Recognition (2026) 113653.

[34] R. Khemchandani, K. Goyal, S. Chandra, Twsvr: regression via twin support vector machine, Neural Networks 74 (2016) 14–21.

[35] P. Ochs, Y. Chen, T. Brox, T. Pock, ipiano: Inertial proximal algorithm for nonconvex optimization, SIAM Journal on Imaging Sciences 7 (2014) 1388–1419.

[36] P. S. Bradley, O. L. Mangasarian, Feature selection via concave minimization and support vector machines., in: ICML, volume 98, 1998, pp. 82–90.

[37] X. Peng, Tpmsvm: a novel twin parametric-margin support vector machine for pattern recognition, Pattern recognition 44 (2011) 2678–2692.

[38] X. Peng, Tsvr: an efficient twin support vector machine for regression, Neural Networks 23 (2010) 365–372.

[39] M. Singla, D. Ghosh, K. Shukla, W. Pedrycz, Robust twin support vector regression based on rescaled hinge loss, Pattern Recognition 105 (2020) 107395.

[40] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine learning research 7 (2006) 1–30.